

Task-Aware Prompt Calibration for Generating Synthetic Novice Python Code

Aidil Muslim
amus790@aucklanduni.ac.nz
University of Auckland
Auckland, New Zealand

ABSTRACT

Large language models offer a potential source of synthetic code for evaluating educational analysis and feedback systems. However, synthetic code may not reproduce the stylistic and structural defects found in authentic novice submissions. This study proposes a task-aware, iterative prompt-calibration approach for generating functionally correct Python code with realistic novice-style defects. Authentic first-year computer science (CS1) student submissions will be analysed to classify defects as task-independent or task-dependent. The classification will build target defect profiles that lead to the prompt construction and refinement. A prototype will compare several prompting approaches through functional correctness tests, profile similarity comparison, and category coverage. This research aims to build a reusable method for producing synthetic novice code that is closer to authentic student code.

CCS Concepts

• Social and professional topics~ Professional topics~ Computing education~ Computing education programs~ Computer science education~ CS1

Keywords

novice programming; code style defects; synthetic data; AI code generation; prompt engineering

1. Literature Review

A systematic literature review (SLR) is performed to detect relevant studies in use of large language models (LLMs) for generating and evaluating novice programming code. The review process will follow the software engineering review guidelines proposed by Kitchenham & Charters (2007), while the identification and study selection process are reported in accordance with the PRISMA 2020 framework. The study review contains four main stages: identification, screening, eligibility assessment, and final inclusion (Page et al., 2021). During identification, candidate studies will be gathered from the selected literature sources using predefined search strings. This research only focuses on open-access studies. Duplicate records will then be removed during this process. The remaining studies will undergo title and abstract screening following the criteria defined in Table 1 to determine their relevance to the research scope. Studies retained after this stage will be assessed for empirical evidence supporting SIC criteria through full-text review using predefined exclusion criteria (EC) shown in Table 2. The final set of eligible studies will be analysed to find the

themes and research strengths represented in the heatmap. Peer-reviewed publications and preprints will be distinguished during data extraction and analysis.

Table 1. Screening inclusion criteria (SIC) and exclusion criteria (SEC)

Screening Code	Description
SIC1	Study addresses authentic novice/student programming code, code style, and defects.
SIC2	Study investigates LLM-based code generation, prompting strategies, or generated programming code.
SIC3	Study investigates synthetic, simulated, buggy, or student-like programming code.
SIC4	Study provides evaluation methods relevant to generated-code correctness, quality, similarity, defects, diversity, or realism.
SEC1	Study is unrelated to programming or source code.
SEC2	Study has no relevant novice/student programming context and no transferable method.
SEC3	LLM is used only for tutoring, feedback, grading, explanation, or general educational support.
SEC4	Study provides no relevant evidence concerning code characteristics, generation, or synthetic student-code modelling.

Table 2. Eligibility Exclusion Criteria (EC)

Eligibility Code	Description
EC1	Study is outside the research focus.
EC2	Code generation or relevant code-characteristic evidence is not evident in the study.
EC3	Study primarily focuses on educational outcomes or support rather than code generation, code characteristics, or synthetic-code evaluation.
EC4	Insufficient methodological detail or duplicate/repeated evidence.

1.1 Databases and Search Strategy

The literature search will use the Association for Computing Machinery (ACM) Digital Library, the Institute of Electrical and Electronics Engineers (IEEE) Xplore, and arXiv. ACM Digital Library and IEEE Xplore are selected as the primary sources because they provide substantial coverage of peer-reviewed

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee, provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

INFOSYS720, The University of Auckland
© 2026 Copyright held by the owner/author(s)

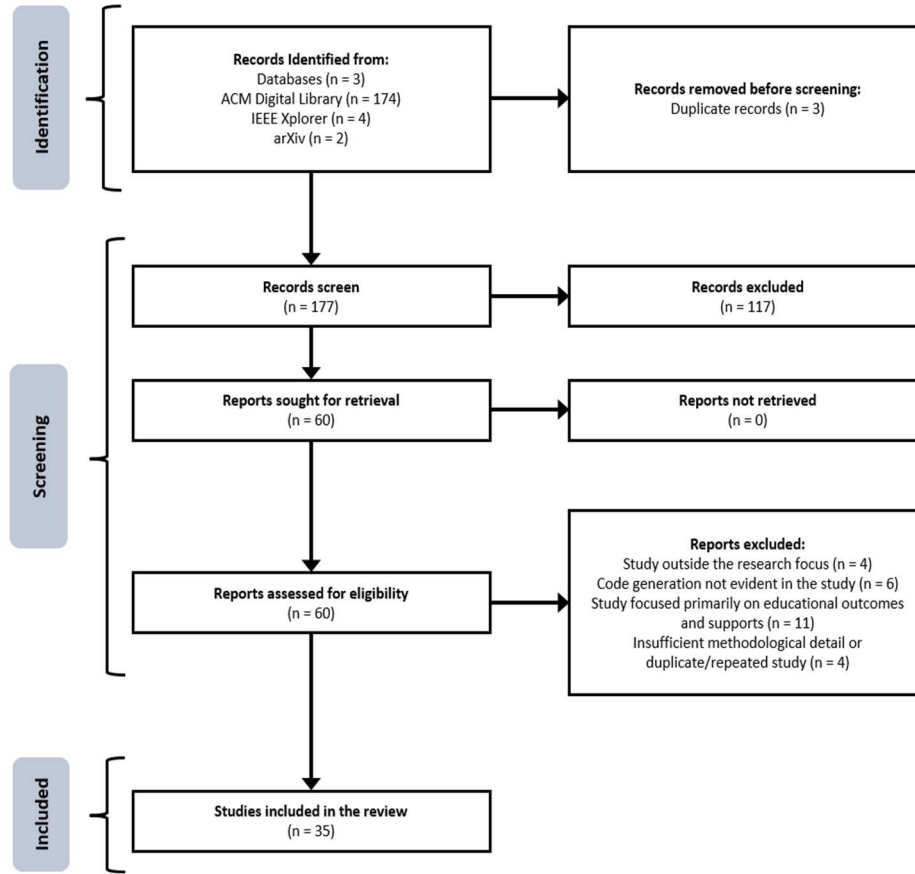


Figure 1. Systematic literature review diagram based on Page (2021)

research in software engineering and artificial intelligence. ACM Digital Library is particularly relevant because research on introductory programming is frequently published through ACM conferences and journals. arXiv will be treated as a supplementary source for identifying recent developments in rapidly evolving areas of LLM research, particularly, emerging work on prompt engineering, synthetic code generation, student simulation, and LLM evaluation. Studies focusing on the mentioned topics may become available as preprints before formal peer-reviewed publication.

The search strategy will be constructed from four principal concept groups: large language models, novice programming, programming defects, and synthetic code generation. Synonymous and related terms within each concept will be connected using the Boolean operator OR to broaden the search result, while the major concept groups will be connected using AND. The target databases support wildcard searching indicated by the asterisk (*). The primary search string will follow the structure:

```

("large language model*" OR LLM* OR "generative AI" OR
GenAI OR GPT*)
AND
("novice programmer*" OR "beginner programmer*" OR
"introductory programming" OR CS1 OR "programming
student*" OR "student code")
AND
("code style" OR "coding style" OR "code quality" OR
"code quality defect*" OR defect* OR bug* OR
"programming pattern*" OR "code pattern*")
AND
("synthetic code" OR "synthetic submission*" OR
"synthetic student*" OR "synthetic program*" OR "code

```

generation" OR "program generation" OR "generate code" OR "generated code")

The literature search identified a total of 180 records. The ACM Digital Library contributed 174 records after applying the content-type filters for Research Articles and Proceedings. The IEEE Xplore search produced 4 records, while arXiv contributed two. A paper by Pădurean, Riegel, et al. (2026) had not been peer-reviewed and was only available in arXiv as a preprint at the time this study was conducted in August 2026. Prior to screening, three duplicate records were removed, leaving 177 records for title and abstract screening. At this stage, 117 records were excluded because they did not satisfy the inclusion criteria presented in Table 1, resulting in 60 studies being retained for full-text eligibility assessment. Following full-text screening, a further 25 studies were excluded based on the eligibility criteria. Consequently, 35 studies were included in the final systematic literature review. The full process diagram presents at Figure 1.

1.2 Search Result

The themes and sub-themes were organised into a logical framework aligned with the purpose of this research. The framework represents a progression from establishing the characteristics of authentic novice code to determining how these characteristics can be reproduced and evaluated using LLMs. The review was based on four main themes, encompassing the set of papers included.

The first theme, novice programming defects, establishes the empirical target by examining the identification, categorisation, and distribution of defects and code-quality characteristics in student submissions. Findings within this theme provide an

empirical basis for determining which novice characteristics may subsequently be represented in the generation process and for constructing target defect profiles associated with programming tasks.

Building on this foundation, LLM-based code generation examines whether generative models provide a suitable mechanism for producing synthetic programming solutions, particularly within educational contexts. This theme concerns the capability of LLMs in generating complete or partial source code based on natural-language instructions, as well as their potential use in programming education for producing examples and exercises.

The third theme, prompt design and generation control, investigates how prompting strategies and constraints can influence generated code and enable the deliberate targeting of particular defect characteristics. This theme considers how prompt structure, specificity, contextual information, and examples can shape the behaviour of LLMs during code generation. It also examines whether structured or defect-oriented prompts can provide greater control over the type and frequency of characteristics introduced into generated solutions. These insights are important for this research because they inform the design of prompting strategies intended to reproduce selected novice programming defect patterns in a controlled and repeatable manner.

Finally, evaluation approaches consider how generated solutions can be assessed through functional correctness, defect-category coverage, code-quality measures, profile similarity, and generation stability. Together, these themes form a logical sequence of target identification, generation, control, and evaluation, providing the conceptual foundation for developing and assessing defect-informed prompting strategies for synthetic novice Python code. Table 3 depicts the themes that were designed for the review.

Table 3. Themes and sub-themes of the literature review

Theme Code	Sub-themes Code
A – Authentic novice code	A1 – Defects/ errors/ misconceptions A2 – Style/ quality/ structure
B – LLM prompting/code generation	B1 – prompt design/ sensitivity B2 – Iterative/ refined prompting
C – Synthetic novice code	C1 – Student-like synthetic generation C2 – Buggy/ incorrect generation
D – Evaluation	D1 – Functional/ code-quality evaluation D2 – Authenticity/ profile similarity

1.2.1 Theme A: Novice programmer defects

Studies within the novice programming defects theme demonstrate that student code contains recurring characteristics that extend beyond functional correctness. Fwa (2024) identifies common programming errors and misconceptions, while Denzler et al. (2024) show that unusual coding-style characteristics can distinguish submissions from expected CS1 patterns. Static analysis further reveals differences in code quality and programming practices, supporting the examination of structural characteristics alongside correctness (Haindl & Weinberger, 2024). Other work broadens assessment by considering efficiency (Tao et al., 2024) and semantic program flow (Zhang et al., 2024), demonstrating that similar outputs may result from substantially different implementations. Approaches based on knowledge components and LLM-assisted assessment additionally provide mechanisms for identifying specific problems within student

solutions (Grandel et al., 2025; Qi et al., 2025). Together, these studies support representing novice programming behaviour through multiple observable defect and quality characteristics. For this research, such characteristics provide a basis for defining defect categories that can guide the controlled generation of synthetic novice code.

Theme A distinguishes between novice programming defects and broader code-quality characteristics. A1 captures recurring errors, misconceptions, and conceptual weaknesses. Studies reflect these topics are in Fwa (2024) and Qi et al. (2025). A2 focuses on style, structure, complexity, and semantic quality, most strongly associated with Tao et al. (2024), Grandel et al. (2025), Denzler et al. (2024), Haindl & Weinberger (2024), and Zhang et al. (2024).

1.2.2 Theme B: LLM-based code generation

Studies within this theme demonstrate that LLMs can generate workable programming solutions from natural-language prompts, including introductory programming tasks (Denny et al., 2023; Finnie-Ansley et al., 2022; Reeves et al., 2023). Much of this work evaluates whether generated code satisfies functional requirements and how prompt formulation influences solution quality. Later studies extend code generation into educational settings, where students interact with LLMs through prompting, code editing, planning, explanation, and dialogue-based problem solving (Kerslake et al., 2024; Nguyen et al., 2024; Pădurean et al., 2025; Pădurean, Gotovos, et al., 2026; Yeh et al., 2025). Other studies examine LLM support for program planning, comprehension, and novice programming practices rather than solution generation alone (Krishnamurthi et al., 2025; Rivera et al., 2024; Smith et al., 2024).

The sub-themes, therefore, distinguish general LLM code-generation capability from its application in novice and educational contexts. The literature in this theme establishes LLMs as capable programming generators, but provides limited evidence that generated solutions systematically reproduce authentic novice characteristics, leaving this as an important gap for the present research.

1.2.3 Theme C: Prompt design and generation control

Studies within this theme examine prompting as a mechanism for shaping LLM-generated code and controlling particular output characteristics. Some research focuses on how prompt structure, examples, contextual information, and task-specific guidance influence generation, while other studies move toward deliberately producing or detecting distinctive code properties. Biderman & Raff (2022) demonstrate that model outputs can be manipulated to exhibit particular characteristics, while more recent work investigates identifiable patterns in LLM-generated submissions (Baek et al., 2026; Hoq et al., 2024; Ramachandra et al., 2026). More relevant to controlled generation, Leinonen et al. (2025) and Denny et al. (2026) explore the production of synthetic educational data, including buggy student-like code, showing that prompts can be used to target desired characteristics rather than merely generate correct solutions. (Pădurean, Riegel, et al., 2026) similarly examines intentionally incorrectly generated code in educational settings.

The sub-themes distinguish prompt design strategies from generation control. Collectively, these studies suggest that prompting can influence both the content and properties of generated programs, providing a basis for translating empirically derived novice defect categories into structured prompts whose ability to reproduce targeted defect profiles can be experimentally evaluated.

1.2.4 Theme D: Evaluation approaches

Studies within this theme evaluate programming solutions using both functional and qualitative criteria. Functional validation commonly assesses whether generated or student-written code satisfies assignment requirements and produces correct outputs, as reflected in evaluations of LLM-generated programming solutions and autograding approaches (Cipriano & Alves, 2023, 2024; Pang & Vahid, 2024). Other studies extend assessment beyond correctness by considering code quality, structural characteristics, refactoring outcomes, rubric-based criteria, and the identification of defects or problematic patterns (Andleeb et al., 2025; Carneiro Oliveira et al., 2025; Pathak et al., 2025; Kumar et al., 2024). Zuzarte Reis Claver et al. (2026) further illustrate how characteristics across collections of submissions can be compared to identify broader programming patterns.

The sub-themes differ in functional validation from defect- or profile-based assessment. For the present research, both are required: functional tests determine whether synthetic programs remain valid solutions, while defect-category coverage and profile similarity assess whether targeted novice characteristics are reproduced. Repeated generations can additionally be compared to determine whether these characteristics are produced consistently rather than appearing by chance.

1.2.5 Literature heatmap

Following the thematic analysis, a heatmap was developed to provide a comparative overview of how the included studies address the identified themes and sub-themes. The purpose of the heatmap is to make the distribution of research attention across the literature more visible and to highlight areas of strong, moderate, or limited coverage. In this review, the papers were mapped against the four main themes of novice programming defects, LLM-based code generation, prompt design and generation control, and evaluation approaches, together with their respective sub-themes. This provides a concise visual synthesis of the literature while supporting comparison across studies beyond a narrative discussion.

Each paper was assessed according to the extent to which it addressed the relevant sub-themes, rather than being ranked by its overall methodological quality. Strong coverage indicates that a sub-theme forms a substantial part of the study, moderate coverage reflects partial or supporting attention, and limited coverage indicates only minor consideration. Areas with little or no coverage are also shown to reveal gaps across the body of literature. This approach helps identify not only the strengths of individual studies but also the degree of integration between novice-defect research, LLM-based generation, prompting strategies, and evaluation methods.

The heatmap, shown in Table 4, is therefore used as a synthesis tool to support the identification of the research gap. In particular, it allows the review to examine whether existing studies connect empirical knowledge of novice programming defects with controlled LLM generation and systematic evaluation of generated code. Patterns of weak or fragmented coverage across these areas provide evidence for the need to investigate defect-informed prompting strategies that can generate synthetic Python solutions with characteristics that more closely reflect authentic novice submissions.

1.3 Literature Synthesising

The systematic literature review reveals four complementary areas of knowledge concerning the generation and evaluation of novice-like programming code. First, research on authentic novice

programming establishes that student submissions contain recurring errors, misconceptions, and structural or stylistic characteristics that are not fully captured by functional correctness alone (Denzler et al., 2024; Fwa, 2024; Haindl & Weinberger, 2024; Zhang et al., 2024). These studies provide an empirical basis for describing novice programming behaviour through observable defect and code-quality characteristics. However, the literature varies in how these characteristics are represented, ranging from individual errors and misconceptions to broader measures of style, complexity, semantic flow, and quality. This suggests that novice code is better represented as a combination of characteristics or a profile rather than through a single indicator.

Second, studies of LLM code generation demonstrate that generative models can produce solutions for introductory programming problems from natural-language instructions (Denny et al., 2023; Finnie-Ansley et al., 2022; Reeves et al., 2023). Educational research has further examined how novices interact with such systems through prompting, code editing, planning, explanation, and dialogue-based problem solving (Kerslake et al., 2024; Nguyen et al., 2024; Pădurean et al., 2025; Pădurean, Gotovos, et al., 2026). Together, these findings establish LLMs as a viable mechanism for producing programming solutions. Nevertheless, successful task completion does not demonstrate that the generated code resembles authentic novice work. Much of the literature evaluates whether a model can solve a programming problem or support learning, rather than whether its output reproduces empirically observed student characteristics.

Third, prompt-design and generation-control research indicates that LLM outputs can be influenced by task descriptions, contextual information, examples, constraints, and other forms of structured guidance. Work on synthetic educational data and deliberately buggy code is particularly relevant because it shows movement from simply generating correct solutions toward intentionally producing selected characteristics (Denny et al., 2026; Leinonen et al., 2025; Pădurean, Riegel, et al., 2026). However, controlled generation remains only partially connected to empirical novice-code evidence. Existing studies provide limited evidence of a systematic process in which defect categories and their observed occurrence in authentic student submissions are translated into prompts intended to reproduce those characteristics.

The fourth theme concerns evaluation. Existing approaches provide mechanisms for validating functional correctness and assessing qualitative properties such as code quality, structure, defects, and rubric-specific criteria (Andleeb et al., 2025; Cipriano & Alves, 2023, 2024; Pang & Vahid, 2024; Pathak et al., 2025). These perspectives are complementary rather than interchangeable. Functional testing determines whether generated code remains a legitimate solution to the task, whereas defect-based assessment determines whether it exhibits the intended novice characteristics. For synthetic novice-code generation, both are necessary because deliberately introducing a defect should not automatically imply functional failure.

Synthesising these themes reveals a methodological research gap at their intersection. The literature separately establishes observable novice-code characteristics, LLM code-generation capability, prompt-based control, and methods for evaluating programming solutions. However, limited research integrates these elements into an empirically grounded generation process that first derives defect profiles from authentic CS1 submissions, then encodes targeted characteristics into structured prompts, and finally evaluates generated solutions for functional correctness, defect-category coverage, profile similarity, and stability across repeated generations.

Table 4. The heatmap of the included papers based on the specified themes and sub-themes

Citation	Primary Theme	Sub-themes								Total	Coverage
		A1	A2	B1	B2	C1	C2	D1	D2		
Qi et al. (2025)	A	3	1	0	0	0	0	3	1	8	4
Grandel et al. (2025)	A	2	3	0	0	0	0	3	1	9	4
Tao et al. (2024)	A	1	3	0	0	0	0	2	0	6	3
Zhang et al. (2024)	A	2	3	0	0	0	0	2	0	7	3
Fwa (2024)	A	3	1	2	0	0	0	3	1	10	5
Denzler et al. (2024)	A	1	3	0	0	0	0	2	2	8	4
Haindl & Weinberger (2024)	A	1	3	1	0	0	0	3	2	10	5
Yeh et al. (2025)	B	0	1	2	3	1	0	3	1	11	6
Denny et al. (2023)	B	0	1	3	2	1	0	3	1	11	6
Reeves et al. (2023)	B	0	1	3	0	0	0	3	1	8	4
Geng et al. (2026)	B	1	2	2	3	1	1	2	1	13	8
Nguyen et al. (2024)	B	1	1	3	3	1	1	3	1	14	8
Kerslake et al. (2024)	B	0	1	3	1	1	0	2	1	9	6
Pădurean, Gotovos, et al. (2026)	B	1	1	2	3	1	1	3	1	13	8
Rivera et al. (2024)	B	0	1	2	3	1	0	3	1	11	6
Krishnamurthi et al. (2025)	B	0	0	2	1	1	0	3	0	7	4
Pădurean et al. (2025)	B	1	1	3	3	1	1	3	1	14	8
Smith et al. (2024)	B	0	1	3	1	1	0	3	1	10	6
Finnie-Ansley et al. (2022)	B	0	2	3	0	1	0	3	2	11	5
Denny et al. (2026)	C	2	2	2	1	3	3	2	2	17	8
Leinonen et al. (2025)	C	3	2	2	1	3	3	3	3	20	8
Pădurean, Riegel, et al. (2026)	C	2	1	2	3	2	3	3	1	17	8
Duan et al. (2025)	C	2	2	0	0	2	1	3	2	12	6
Biderman & Raff (2022)	C	0	2	1	0	2	0	2	3	10	5
Ramachandra et al. (2026)	C	1	2	2	0	3	0	2	3	13	6
Hoq et al. (2024)	C	0	3	2	0	3	0	2	3	13	5
Back et al. (2026)	C	0	3	1	0	3	0	2	3	12	5
Pang & Vahid (2024)	D	0	3	1	0	2	0	2	3	11	5
Andleeb et al. (2025)	D	1	3	1	0	1	0	3	1	10	6
Cipriano & Alves (2023)	D	1	3	1	0	1	0	3	1	10	6
S Kumar et al. (2024)	D	3	1	2	1	0	1	3	1	12	7
Cipriano & Alves (2024)	D	2	3	1	0	1	1	3	1	12	7
Pathak et al. (2025)	D	2	2	1	0	0	0	3	1	9	5
Zuzarte Reis Claver et al. (2026)	D	1	2	1	0	2	0	2	3	11	6
Carneiro Oliveira et al. (2025)	D	0	3	1	2	0	0	3	1	10	5

Note. Sub-theme cells are coded using a four-point scale: 0 = not addressed (white), 1 = limited coverage (light blue), 2 = moderate coverage (medium blue), and 3 = strong coverage (dark blue). Total represents the sum of the eight sub-theme scores for each paper, with a possible range of 0–24; darker red shading indicates a higher cumulative thematic contribution. Coverage represents the number of sub-themes addressed by a paper (i.e., receiving a score ≥ 1), with a possible range of 0–8; darker red shading indicates broader coverage across the identified sub-themes. Colour intensity, therefore, represents the relative strength or breadth of thematic coverage.

Figure 2 illustrates this gap as the intersection of three bodies of knowledge: authentic novice-code characteristics, LLM generation and prompt control, and functional and defect-profile evaluation. The central intersection represents the research opportunity

addressed in this study: a method for generating functionally valid synthetic CS1 code whose defect characteristics are deliberately controlled and evaluated against empirical novice profiles.

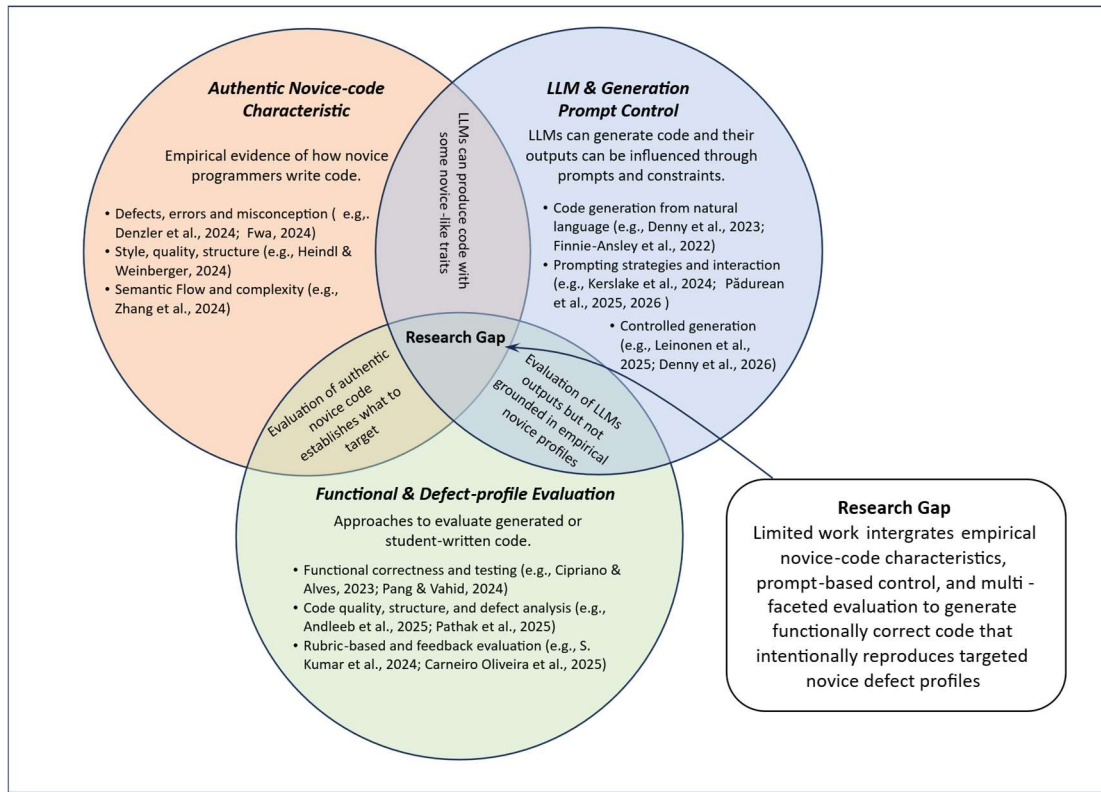


Figure 2. The research gap from the literature review is represented in a Venn diagram.

2. System Review

Following the SLR synthesis, a Systematic Systems Review (SSR) was conducted to examine existing systems, platforms, and technologies that could support the proposed generation and evaluation of synthetic novice Python code. The review focused on technologies that provide at least one of the functions required by the proposed artefact: LLM-based code generation, prompt and inference control, functional validation, educational code assessment, or structural code analysis. Representative systems were selected across these functions, including Google Gemini API, Hugging Face Inference Providers, Ollama, PrairieLearn, pytest, and Python analysis technologies. Rather than identifying a single system capable of solving the research problem, the review examines the affordances and limitations of existing technologies and determines how they could contribute to an integrated defect-informed generation workflow. Table 5 represents the system reviews for each technology.

The first group concerns LLM generation platforms. Google Gemini Application Programming Interface (API) provides programmatic access to Gemini models through standard development kits (SDKs) and Representational State Transfer (REST) APIs, and supports system instructions for guiding model behaviour. Its generation APIs can also be used for code-oriented tasks, making it suitable for automating repeated Python code generation experiments (Google, n.d.). Hugging Face Inference Providers offers a different model-access approach by providing a unified interface to hundreds of models through Python and JavaScript clients, allowing researchers to experiment with different model providers without constructing a separate interface for each one (Hugging Face, n.d.). Ollama represents a local deployment alternative, exposing an API for interacting with models running on a local machine and providing official Python

and JavaScript libraries (Ollama, n.d.). These platforms, therefore, provide different approaches to the generation layer—managed cloud models, multi-provider model access, and local model execution. However, based on their documented capabilities, the platforms provide general prompt and generation mechanisms rather than a built-in method for translating empirical novice-defect profiles into controlled programming outputs. This limitation means that defect-oriented generation logic must be developed externally as part of the proposed artefact.

The second group concerns functional and educational assessment. PrairieLearn provides an educational assessment environment capable of automatically grading submitted programs. Its external grading mechanism can execute custom grading scripts in specified environments, while a dedicated Python autograder is available for Python submissions (PrairieLearn, n.d.). This demonstrates that automated execution of student code is already well supported within programming-education systems. At a lighter implementation level, pytest provides assertion-based testing and parameterisation, allowing a generated solution to be tested against multiple input and expected-output combinations (pytest Development Team, n.d.). These technologies are suitable for determining whether generated programs continue to satisfy the original programming task. Their limitation for the present research is that successful functional tests establish behavioural correctness but do not determine whether a program exhibits the intended novice defect characteristics.

The final group supports structural and defect-oriented analysis. Python's built-in `ast` module enables source code to be parsed into an abstract syntax tree (AST) and provides mechanisms such as `NodeVisitor` for programmatically traversing code structures (Python Software Foundation, n.d.). This provides flexibility for implementing custom rules corresponding to defect categories established from empirical novice submissions. Ruff provides

Table 5. Systematic system review of relevant technologies

System/ technologies	Primary function	Relevant features/affordances	Limitation of the research	Potential role
Google Gemini API	LLM generation	Programmatic generation, REST/SDK access, system instructions	No documented native mechanism for empirical novice-defect profiles	Primary code-generation engine
Hugging Face Inference Providers	Model Access	Unified access to many hosted models; Python client	Model behaviour and capability vary; defect control must be externally defined	Alternative/model comparison platform
Ollama	Local LLM execution	Local API, Python integration, configurable model execution	Dependent on local hardware/model capability; no novice-defect framework	Local-generation alternative
PrairieLearn	Educational code assessment	Python autograder, external/custom grading	Designed primarily to assess submissions rather than generate novice-like code	Reference educational assessment system
pytest	Functional validation	Assertions, parametrised test cases, automated testing	Correctness testing does not measure novice characteristics	Validate functional correctness
Python AST / Ruff	Structural analysis	AST traversal, custom structural detection, predefined lint rules	Existing lint rules may not correspond directly to empirical novice defects	Detect and quantify target defects

complementary static-analysis capability through a large collection of Python linting rules, including rules derived from several established linting tools (Astral Software, n.d.). However, general-purpose linting rules are designed primarily around conventional Python quality and style concerns; therefore, their correspondence with the specific novice-defect taxonomy used in this research would need to be established rather than assumed.

Overall, the SSR indicates that the technological components required by the proposed research already exist, but their capabilities remain distributed across separate systems. LLM platforms provide generation and prompt control, testing and educational assessment systems provide functional validation, and static or structural analysis technologies provide mechanisms for examining source-code characteristics. The systems gap is therefore not the absence of individual generation or evaluation technologies, but the absence of an integrated mechanism that converts empirically derived novice-defect profiles into controlled LLM prompts and then evaluates the resulting programs against both functional and defect-oriented criteria. This gap provides the technological grounding for an artefact integrating defect profiles, prompt construction, LLM generation, functional testing, defect detection, and profile comparison.

3. Gap & Kernel-theory Identification

The combined findings of the SLR and SSR identify complementary research and systems gaps concerning the generation of synthetic novice Python code. The SLR established that LLMs are capable of solving introductory programming problems (Finnie-Ansley et al., 2022), while prompt formulation can influence the success and characteristics of generated solutions (Denny et al., 2023; Reeves et al., 2023). Most directly, Leinonen

et al. (2025) demonstrated that LLMs can generate synthetic buggy programming submissions and compared their test-case failure distributions with those of authentic student submissions. These findings establish the feasibility of synthetic novice-code generation, but the reviewed literature provides limited integration between empirically identified novice defects, controlled prompting, and evaluation of whether generated code reproduces an explicit target defect profile. Therefore, the resulting research gap concerns the controlled generation of functionally valid synthetic code according to empirically grounded novice-defect characteristics.

The SSR identifies a corresponding systems gap. Existing LLM platforms provide programmatic generation and prompt control, testing technologies support functional validation, and static or structural analysis technologies enable source-code characteristics to be detected. However, these capabilities exist as separate technological components. None of the reviewed systems provides an integrated process that begins with an empirical novice-defect profile, translates that profile into generation constraints, generates synthetic programs, and subsequently compares their observed defect characteristics with the intended profile. The systems gap is therefore primarily one of integration and defect-informed generation control, rather than an absence of the underlying technologies.

Representation Theory is adopted as the primary justificatory theory for addressing these gaps. The theory is based on the premise that an information system represents phenomena in a real-world domain and that the faithfulness of this representation is important to the usefulness of the system (Recker et al., 2019; Wand & Weber, 1988).

Table 6. Problem, issues, requirement of the research presented in a table

Problems	Issues	Requirements
Lack of a systematic classification of task-independent and task-dependent code-quality defects in authentic novice CS1 Python submissions.	Issue related to the inconsistent representation of novice-code characteristics.	Classify defects according to cross-task occurrence.
	Issue related to the limited distinction between task-independent and task-dependent defect occurrence patterns	Construct empirical task-aware defect profiles.
Lack of an integrated method for translating empirical defect profiles into iterative, task-aware LLM prompts while preserving functional correctness and authentic defect distributions.	Empirical defect profiles are not systematically translated into adaptive prompting.	Translate defect profiles into task-aware prompting and evaluate both correctness and distribution alignment.
	Generation, functional testing, defect analysis, and refinement are technologically separate.	Support iterative prompt refinement and comparison with a non-adaptive baseline.

Table 7. Questions, objectives, and artefacts of the research presented in a table

Questions	Objectives	Artefacts
RQ1: How can code-quality defect types in authentic CS1 Python submissions be classified as task-independent and task-dependent based on their occurrence patterns across programming tasks?	Develop an empirical classification method that distinguishes task-independent and task-dependent code-quality defects based on their occurrence patterns across CS1 programming tasks.	<i>Conceptual artefact:</i> A task-aware defect classification and profiling framework that represents novice-code characteristics according to their occurrence across programming tasks.
RQ2: How can empirical profiles of task-independent and task-dependent defects be incorporated into a task-aware prompting approach for generating functionally correct synthetic novice code?	Develop a task-aware prompting approach that translates empirical task-independent and task-dependent defect profiles into generation constraints for producing functionally correct synthetic novice Python code.	<i>Conceptual artefact</i> defines the defect-profile-to-prompt transformation. <i>System artefact</i> operationalises the prompting approach through LLM-based code generation and functional validation.
RQ3: To what extent does iterative, task-aware prompt refinement improve the alignment between the defect distributions of synthetic code and authentic CS1 student submissions compared with a non-adaptive prompting approach?	Evaluate whether iterative task-aware prompt refinement improves defect-distribution alignment relative to a non-adaptive prompting baseline while maintaining functional correctness.	<i>System artefact:</i> An experimental prototype supporting iterative prompt refinement, LLM generation, functional testing, defect detection, distribution comparison, and baseline evaluation.

In this research, authentic CS1 Python submissions form the real-world domain, while their selected observable defect characteristics initiate the phenomena to be represented. The conceptual artefact abstracts these characteristics into defect categories and task-specific defect profiles. Rather than attempting to model the complete cognitive behaviour of novice programmers, the profile represents selected observable characteristics of their submitted programs.

The system artefact realises this conceptual representation. Defect-profile characteristics are translated into prompt constraints,

which guide an LLM in generating synthetic Python submissions. The generated programs then form synthetic representations whose detected defect characteristics can be compared with the empirical target. Hence, Representation Theory provides theoretical justification for evaluating defect-category coverage and profile similarity, as these measures assess correspondence between the generated representation and the target phenomenon. Functional correctness remains a separate evaluation criterion because representational similarity does not guarantee that the generated program continues to satisfy the original programming task. This

interpretation is consistent with Representation Theory's emphasis on faithfully representing relevant real-world phenomena.

Design Science Research (DSR) provides the overarching paradigm for constructing and evaluating these artefacts. DSR emphasises the creation and evaluation of purposeful artefacts for identified problems (Hevner et al., 2004). Thus, Representation Theory explains what should be represented and how representational adequacy can be assessed, while the SLR establishes the empirical and methodological knowledge required for the representation, and the SSR identifies the technologies through which it can be operationalised.

These findings are consolidated in the Problem, Issue, Requirement, Question, Objective, and Artefact (PIRQOA) framework presented in Table 6 and Table 7. The single research problem is decomposed into four issues concerning defect classification, generation control, evaluation, and systems integration. These issues lead to four corresponding artefact requirements: task-aware defect profiling, profile-informed prompt construction, combined functional and defect-based validation, and iterative refinement. The three research questions then guide the objectives and define the conceptual and system artefacts required to address the identified gaps.

References

- Andleeb, S., Kantorski, B., & Carver, J. (2025). ChatGPT in Introductory Programming: Counterbalanced Evaluation of Code Quality, Conceptual Learning, and Student Perceptions. *Proceedings of the 26th ACM Annual Conference on Cybersecurity & Information Technology Education, SIGCITE '25*, 180–186. <https://doi.org/10.1145/3769694.3771128>
- Astral Software. (n.d.). *Rules*. Retrieved August 12, 2026, from <https://docs.astral.sh/ruff/rules/>
- Baek, J., Yamazaki, T., Morihata, A., Mori, J., Yamakata, Y., Taura, K., & Chiba, S. (2026). LLM-Based Explainable Detection of LLM-Generated Code in Python Programming Courses. *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1, SIGCSE TS 2026*, 80–86. <https://doi.org/10.1145/3770762.3772605>
- Biderman, S., & Raff, E. (2022). Fooling MOSS Detection with Pretrained Language Models. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management, CIKM '22*, 2933–2943. <https://doi.org/10.1145/3511808.3557079>
- Carneiro Oliveira, E., Keuning, H., & Jeuring, J. (2025). Uncovering Behavioral Patterns in Student–LLM Conversations during Code Refactoring Tasks. *Proceedings of the 25th Koli Calling International Conference on Computing Education Research, Koli Calling '25*. <https://doi.org/10.1145/3769994.3770000>
- Cipriano, B. P., & Alves, P. (2023). GPT-3 vs Object Oriented Programming Assignments: An Experience Report. *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2023*, 61–67. <https://doi.org/10.1145/3587102.3588814>
- Cipriano, B. P., & Alves, P. (2024). LLMs Still Can't Avoid Instanceof: An Investigation Into GPT-3.5, GPT-4 and Bard's Capacity to Handle Object-Oriented Programming Assignments. *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, ICSE-SEET '24*, 162–169. <https://doi.org/10.1145/3639474.3640052>
- Denny, P., Kumar, V., & Giacaman, N. (2023). Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2023*, 1136–1142. <https://doi.org/10.1145/3545945.3569823>
- Denny, P., Riegel, K., Luxton-Reilly, A., Leinonen, J., Prather, J., & MacNeil, S. (2026). Enabling Postgraduate Projects in Computing Education through Synthetic Research Data Generation. *Proceedings of the 28th Australasian Computing Education Conference, ACE '26*, 85–94. <https://doi.org/10.1145/3786228.3786241>
- Denzler, B., Vahid, F., Pang, A., & Salloum, M. (2024). Style Anomalies Can Suggest Cheating in CS1 Programs. *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2024*, 381–387. <https://doi.org/10.1145/3649217.3653626>
- Duan, Z., Fernandez, N., Hicks, A., & Lan, A. (2025). Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks. *Proceedings of the 15th International Learning Analytics and Knowledge Conference, LAK '25*, 238–248. <https://doi.org/10.1145/3706468.3706500>
- Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. *Proceedings of the 24th Australasian Computing Education Conference, ACE '22*, 10–19. <https://doi.org/10.1145/3511861.3511863>
- Fwa, H. L. (2024). Experience Report: Identifying common misconceptions and errors of novice programmers with ChatGPT. *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, ICSE-SEET '24*, 233–241. <https://doi.org/10.1145/3639474.3640059>
- Geng, F., Shah, A., Li, H., Mulla, N., Swanson, S., Soosai Raj, G., Zingaro, D., & Porter, L. (2026). Exploring Student-AI Interactions in Vibe Coding. *Proceedings of the 28th Australasian Computing Education Conference, ACE '26*, 45–54. <https://doi.org/10.1145/3786228.3786236>
- Google. (n.d.). *Text Generation*. Retrieved August 12, 2026, from https://ai.google.dev/gemini-api/docs/text-generation#python_7

- Grandel, S., Schmidt, D., & Leach, K. (2025). Applying Large Language Models to Enhance the Assessment of Java Programming Assignments. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, FSE Companion '25*, 789–799. <https://doi.org/10.1145/3696630.3727236>
- Haindl, P., & Weinberger, G. (2024). Does ChatGPT Help Novice Programmers Write Better Code? Results From Static Code Analysis. *IEEE Access*, 12, 114146–114156. <https://doi.org/10.1109/ACCESS.2024.3445432>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Q.*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Hoq, M., Shi, Y., Leinonen, J., Babalola, D., Lynch, C., Price, T., & Akram, B. (2024). Detecting ChatGPT-Generated Code Submissions in a CS1 Course Using Machine Learning Models. *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1, SIGCSE 2024*, 526–532. <https://doi.org/10.1145/3626252.3630826>
- Hugging Face. (n.d.). *Inference Providers*. Retrieved August 12, 2026, from <https://huggingface.co/docs/inference-providers/index>
- Kerslake, C., Denny, P., Smith, D. H., Prather, J., Leinonen, J., Luxton-Reilly, A., & MacNeil, S. (2024). Integrating Natural Language Prompting Tasks in Introductory Programming Courses. *Proceedings of the 2024 on ACM Virtual Global Computing Education Conference V. 1, SIGCSE Virtual 2024*, 88–94. <https://doi.org/10.1145/3649165.3690125>
- Kitchenham, B., & Charters, S. (2007). *Guidelines for Performing Systematic Literature Reviews in Software Engineering* (Number EBSE-2007-01).
- Krishnamurthi, S., Thießen, T., & Vahrenhold, J. (2025). Porpoise: An LLM-Based Sandbox for Novices to Practice Writing Purpose Statements. *Proceedings of the 2025 ACM SIGPLAN International Symposium on SPLASH-E, SPLASH-E '25*, 75–89. <https://doi.org/10.1145/3758317.3759683>
- Leinonen, J., Denny, P., Kiljunen, O., MacNeil, S., Sarsa, S., & Hellas, A. (2025). LLM-itation is the Sincerest Form of Data: Generating Synthetic Buggy Code Submissions for Computing Education. *Proceedings of the 27th Australasian Computing Education Conference, ACE '25*, 56–63. <https://doi.org/10.1145/3716640.3716647>
- Nguyen, S., Babe, H. M., Zi, Y., Guha, A., Anderson, C. J., & Feldman, M. Q. (2024). How Beginning Programmers and Code LLMs (Mis)read Each Other. *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems, CHI '24*. <https://doi.org/10.1145/3613904.3642706>
- Ollama. (n.d.). *Introduction*. Retrieved August 12, 2026, from <https://docs.ollama.com/api/introduction>
- Pădurean, V.-A., Denny, P., Gotovos, A., & Singla, A. (2025). Prompt Programming: A Platform for Dialogue-based Computational Problem Solving with Generative AI Models. *Proceedings of the 30th ACM Conference on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2025*, 458–464. <https://doi.org/10.1145/3724363.3729094>
- Pădurean, V.-A., Gotovos, A., Ghosh, A., Denny, P., Leinonen, J., Luxton-Reilly, A., Prather, J., & Singla, A. (2026). Interleaving Natural Language Prompting with Code Editing for Solving Programming Tasks with Generative AI Models. *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2026*, 273–279. <https://doi.org/10.1145/3803400.3809312>
- Pădurean, V.-A., Riegel, K., Gotovos, A., Mahapatra, J., Ghosh, A., Denny, P., Leinonen, J., Prather, J., & Singla, A. (2026). *When AI Is Wrong on Purpose: How Students Respond to Buggy GenAI Code*. <https://doi.org/10.48550/arXiv.2607.05068>
- Page, M. J., McKenzie, J. E., Bossuyt Patrick M. and Boutron, I., Hoffmann, T. C., Mulrow Cynthia D. and Shamseer, L., Tetzlaff, J. M., Akl Elie A. and Brennan, S. E., Chou, R., Glanville Julie and Grimshaw, J. M., Hróbjartsson Asbjørn and Lalu, M. M., Li, T., Loder Elizabeth W. and Mayo-Wilson, E., McDonald, S., McGuinness Luke A. and Stewart, L. A., Thomas, J., Tricco Andrea C. and Welch, V. A., Whiting, P., & Moher, D. (2021). The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews. *BMJ*, 372, n71. <https://doi.org/10.1136/bmj.n71>
- Pang, A., & Vahid, F. (2024). ChatGPT and Cheat Detection in CS1 Using a Program Autograding System. *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2024*, 367–373. <https://doi.org/10.1145/3649217.3653558>

- Pathak, A., Gandhi, R., Uttam, V., Ramamoorthy, A., Ghosh, P., Jindal, A. R., Verma, S., Mittal, A., Ased, A., Khatri, C., Nakka, Y., Devansh, Challa, J. S., & Kumar, D. (2025). Rubric Is All You Need: Improving LLM-Based Code Evaluation With Question-Specific Rubrics. *Proceedings of the 2025 ACM Conference on International Computing Education Research V.1, ICER '25*, 181–195. <https://doi.org/10.1145/3702652.3744220>
- PrairieLearn. (n.d.). *Python Autograder*. Retrieved August 12, 2026, from <https://docs.prairielearn.com/python-grader/>
- pytest Development Team. (n.d.). *How to parametrize fixtures and test functions*. Retrieved August 12, 2026, from <https://docs.pytest.org/en/stable/how-to/parametrize.html>
- Python Software Foundation. (n.d.). *ast — Abstract syntax trees*. Retrieved August 12, 2026, from <https://docs.python.org/3/library/ast.html>
- Qi, L., Zamfirescu-Pereira, J. D., Kim, T., Hartmann, B., DeNero, J., & Norouzi, N. (2025). A Knowledge-Component-Based Methodology for Evaluating AI Assistants. *Proceedings of the ACM Global Computing Education Conference 2025 - Volume 1, CompEd 2025*, 78–84. <https://doi.org/10.1145/3736181.3747167>
- Ramachandra, A., Chaudhary, S., Tran, J., Desai, R., Pang, A., & Salloum, M. (2026). Detecting AI-Generated Code in Introductory Programming Courses. *Proceedings of the 57th ACM Technical Symposium on Computer Science Education V.1, SIGCSE TS 2026*, 894–900. <https://doi.org/10.1145/3770762.3772522>
- Recker, J., Indulska, M., Green, P. F., Burton-Jones, A., & Weber, R. (2019). Information systems as representations: A review of the theory and evidence. *Journal of the Association for Information Systems*, 20(6), 735–786. <https://doi.org/10.17705/1jais.00550>
- Reeves, B., Sarsa, S., Prather, J., Denny, P., Becker, B. A., Hellas, A., Kimmel, B., Powell, G., & Leinonen, J. (2023). Evaluating the Performance of Code Generation Models for Solving Parsons Problems With Small Prompt Variations. *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2023*, 299–305. <https://doi.org/10.1145/3587102.3588805>
- Rivera, E., Steinmaurer, A., Fisler, K., & Krishnamurthi, S. (2024). Iterative Student Program Planning using Transformer-Driven Feedback. *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, ITiCSE 2024*, 45–51. <https://doi.org/10.1145/3649217.3653607>
- S Kumar, S., Adam Lones, M., Maarek, M., & Zantout, H. (2024). Investigating the Proficiency of Large Language Models in Formative Feedback Generation for Student Programmers. *Proceedings of the 1st International Workshop on Large Language Models for Code, LLM4Code '24*, 88–93. <https://doi.org/10.1145/3643795.3648380>
- Smith, D. H., Denny, P., & Fowler, M. (2024). Prompting for Comprehension: Exploring the Intersection of Explain in Plain English Questions and Prompt Writing. *Proceedings of the Eleventh ACM Conference on Learning @ Scale, L@S '24*, 39–50. <https://doi.org/10.1145/3657604.3662039>
- Tao, Y., Chen, W., Ye, Q., & Zhao, Y. (2024). Beyond Functional Correctness: An Exploratory Study on the Time Efficiency of Programming Assignments. *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, ICSE-SEET '24*, 320–330. <https://doi.org/10.1145/3639474.3640065>
- Wand, Y., & Weber, R. (1988). An Ontological Analysis of Some Fundamentals Information Systems Concepts. *Journal of the Association for Information Systems*, 35. <http://aisel.aisnet.org/icis1988/35>
- Yeh, T. Y., Tran, K., Gao, G., Yu, T., Fong, W. O., & Chen, T.-Y. (2025). Bridging Novice Programmers and LLMs with Interactivity. *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 1, SIGCSETS 2025*, 1295–1301. <https://doi.org/10.1145/3641554.3701867>
- Zhang, A. G., Tang, X., Oney, S., & Chen, Y. (2024). CFlow: Supporting Semantic Flow Analysis of Students' Code in Programming Problems at Scale. *Proceedings of the Eleventh ACM Conference on Learning @ Scale, L@S '24*, 188–199. <https://doi.org/10.1145/3657604.3662025>
- Zuzarte Reis Claver, J. M., Tajdini, M., & Verano Merino, M. (2026). The Clash of Codes: From Peer-to-Peer Duplication to AI-Generation in Introductory Programming Assignments. *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering, ICSE-SEET '26*, 59–69. <https://doi.org/10.1145/3786580.3786955>

